Evaluation

MMLU · HumanEval · MT-Bench · LLM-as-Judge · Execution Accuracy

Spider · ROUGE · BLEU · lm-evaluation-harness · Regression Testing

WEEK 2 · DAY 12 **Friday, July 17 2026** **Day 12 of 84**

#	Topic	What You Learn
01	Evaluation Taxonomy	Automated benchmarks vs LLM-as-judge vs human eval
02	MMLU	57-subject MCQ; world knowledge; 25% baseline, 86% GPT-4
03	HumanEval + HumanEval+	pass@k; unit test execution; 80x harder test cases
04	GSM8K	Grade school math; chain-of-thought; multi-step reasoning
05	Spider	Text-to-SQL; 10K questions; execution accuracy scoring
06	MT-Bench + LLM-as-Judge	GPT-4 judge; 8 categories; multi-turn quality; rubric scoring
07	Execution Accuracy	SQL correct iff result rows equal ground truth
08	lm-evaluation-harness	EleutherAI; 200+ tasks; vLLM backend; full eval pipeline
09	Regression Testing	Curated test cases; CI gate; catch regressions pre-deploy
10	ROUGE and BLEU	ROUGE for summarization; BLEU for translation; not for SQL
11	FAANG in Production	Meta 5-tier eval; Google BIG-Bench; Amazon A/B; Netflix holdout
12	Text-to-SQL Thread	Capstone Piece 4: full evaluation harness for SQL model

TEXT-TO-SQL CAPSTONE THREAD — Piece 4 of 8: the Evaluation Harness. Today you build the system that measures execution accuracy across difficulty tiers (easy/medium/hard), runs LLM-as-judge for SQL quality, and serves as the CI gate before any model update reaches production.

Section 1 — The Evaluation Taxonomy

Evaluation has three tiers. They are complementary — not interchangeable. Automated benchmarks are fast but miss quality nuance. LLM-as-judge captures nuance but costs money. Human evaluation is the ground truth but is expensive. Use all three; the pyramid tells you when.

```
EVALUATION TYPES:

1. AUTOMATED BENCHMARKS
   Standardized datasets with known correct answers.
   Fast, cheap, reproducible, comparable across papers and models.
   Risk: benchmark contamination (model may have seen test data in training).
   Examples: MMLU, HumanEval, GSM8K, Spider (SQL execution accuracy)

2. LLM-AS-JUDGE
   A stronger model evaluates responses from the tested model.
   Captures quality dimensions automated benchmarks miss: coherence,
   style, depth, reasoning quality.
   Risk: judge model bias, positional bias, cost per evaluation.
   Examples: MT-Bench (GPT-4 judge), AlpacaEval, Arena-Hard, custom rubrics

3. HUMAN EVALUATION
   Humans rate model outputs on defined criteria.
   Ground truth — catches all failure modes, including ones you didn't anticipate.
   Risk: expensive ($5-20 per annotation), slow (days-weeks), inter-annotator disagreement.
   Examples: Chatbot Arena, internal red-teaming, A/B testing in production
```

> *WHY EVALUATION IS THE MOST UNDERINVESTED AREA: every major AI incident (wrong medical advice, hallucinated legal citations, biased hiring output) was an evaluation failure before it was a production failure. The rule: if you cannot measure it, you cannot improve it — and you cannot safely deploy it.*

Section 2 — MMLU: World Knowledge Benchmark

MMLU (Hendrycks et al., 2020) tests world knowledge across 57 subjects: mathematics, history, law, medicine, computer science, ethics, and more. Each question is 4-choice multiple choice. It is the most widely cited general-purpose LLM benchmark.

```
MMLU structure:
  57 subjects x ~200 questions = ~14,000 total questions
  Format: 4-choice MCQ
  Scoring: accuracy (fraction correct)
  Random baseline: 25% (always check: is the model better than random?)

Example (Professional Law):
  Q: Which is NOT a recognized exception to the Fourth Amendment warrant requirement?
  A. Exigent circumstances   B. Consent
```

```
  C. Plain view              D. Mere suspicion   <- correct

Score interpretation (5-shot, across all subjects):
  Random baseline:   25%
  GPT-3.5:          ~57%
  LLaMA 3 8B:       ~68%
  LLaMA 3 70B:      ~82%
  GPT-4:            ~86%
  Human expert avg: ~89%

Run with lm-evaluation-harness:
  lm_eval --model hf \
          --model_args pretrained=meta-llama/Llama-3.1-8B-Instruct \
          --tasks mmlu --num_fewshot 5 --batch_size 16 \
          --output_path ./results/mmlu.json
```

After fine-tuning on SQL data, re-run MMLU immediately. A drop > 3% in MMLU indicates catastrophic forgetting — the fine-tuning has damaged general knowledge (Days 9-10).

Section 3 — HumanEval, HumanEval+, MBPP: Code Benchmarks

HumanEval (Chen et al., OpenAI 2021): 164 Python programming problems with docstrings + function signatures. The model completes the function body. Scored by pass@k: probability at least 1 of k samples passes all unit tests.

```
HumanEval problem structure:
  def has_close_elements(numbers: list, threshold: float) -> bool:
      """ Check if any two numbers are closer than threshold.
      >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
      False
      >>> has_close_elements([1.0, 2.8, 3.0], 0.3)
      True
      """
      # Model fills this in; evaluated by running unit tests

pass@k formula:
  pass@1:  accuracy of single sample (temperature=0.2)
  pass@10: P(at least 1 of 10 passes)  (temperature=0.8, more diverse)
  pass@k = 1 - C(n-c, k) / C(n, k)   where n=samples, c=passing samples

Score comparison (pass@1):
  GPT-3.5:         ~48%
  LLaMA 3 8B:      ~62%
  GPT-4:           ~67%
  LLaMA 3 70B:     ~81%
  Code Llama 34B:  ~53%

HumanEval+ (Liu et al., 2023):
  80x more test cases per problem than original HumanEval
```

```
  Models that memorised solutions drop 10-25% on HumanEval+
  Always use HumanEval+ for rigorous code evaluation

MBPP (Mostly Basic Python Problems): 374 problems; crowdsourced; complementary to HumanEval
```

Section 4 — GSM8K: Math Reasoning

GSM8K (Cobbe et al., OpenAI 2021): 8,500 grade school math word problems requiring multi-step arithmetic and logical reasoning. The metric is exact match on the final numeric answer.

```
GSM8K example:
  Problem: Janet's ducks lay 16 eggs per day. She eats 3 for breakfast
           and bakes muffins with 4. She sells the rest at $2 per egg.
           How much does she make daily?

  Chain-of-thought solution:
  Step 1: Eggs eaten or used: 3 + 4 = 7
  Step 2: Eggs remaining: 16 - 7 = 9
  Step 3: Revenue: 9 x $2 = $18
  Answer: $18  <- exact match required

Score comparison (8-shot chain-of-thought):
  GPT-3.5:      57%
  LLaMA 3 8B:   79%   (benefits hugely from CoT prompting)
  LLaMA 3 70B:  93%
  GPT-4:        92%

Key insight: LLaMA 3 8B jumps from ~30% (greedy, no CoT) to 79% (8-shot CoT)
-> Chain-of-thought prompting is mandatory for math tasks
```

Section 5 — Spider: Text-to-SQL Benchmark

Spider (Yu et al., Yale NLP 2018) is the standard benchmark for Text-to-SQL: 10,181 questions across 200 databases and 138 domains. Scored by execution accuracy — the generated SQL must return the same rows as the ground truth SQL.

```
Execution accuracy implementation:
import sqlite3

def execution_accuracy(predicted_sql, ground_truth_sql, db_path):
    conn = sqlite3.connect(db_path)
    try:
        pred = set(tuple(r) for r in conn.execute(predicted_sql).fetchall())
        gold = set(tuple(r) for r in conn.execute(ground_truth_sql).fetchall())
        return pred == gold
    except Exception:
        return False   # syntax error or runtime error = incorrect
    finally:
        conn.close()

Spider score comparison:
  Baseline (random):              ~10%
  GPT-4 (prompting, no FT):       82%
  LLaMA 3 8B + LoRA (Day 9):     85-88%
```

```
  LLaMA 3 8B + Full FT (Day 10): 93-95%
  Best fine-tuned models:        ~96%

Difficulty stratification (LLaMA 3 8B + LoRA):
  easy   (simple SELECT + WHERE):     96%
  medium (JOIN, GROUP BY, HAVING):    89%
  hard   (nested subqueries):         78%
  extra  (INTERSECT, EXCEPT, UNION):  62%
  -> Overall 85% hides the 62% on hard queries
```

ARCHITECT RULE: Never report a single aggregate accuracy number. Always break down by difficulty. A model at 85% overall that fails 38% of complex queries is NOT production-ready for a reporting system. Fix the hard tier before declaring the model production-ready.

Section 6 — MT-Bench and LLM-as-Judge

MT-Bench (Zheng et al., UC Berkeley 2023) evaluates multi-turn conversation quality across 8 categories: writing, roleplay, reasoning, math, coding, extraction, STEM, humanities. GPT-4 scores each response 1-10.

```
MT-Bench structure:
  80 two-turn conversations across 8 categories
  Turn 1: initial question
  Turn 2: follow-up requiring memory of Turn 1 (tests coherence and context)

Example (Reasoning):
  Turn 1: How would you solve the Towers of Hanoi for 4 disks?
  Turn 2: How does the time complexity change if we add 2 more disks?

GPT-4 judge prompt (simplified):
  [Question] {question}
  [Model Response] {response}
  [System] Score the response 1-10. Consider correctness, completeness,
  clarity, depth. Output: [[score]]

MT-Bench scores (out of 10):
  Claude 3 Opus:   9.02
  GPT-4:           8.99
  LLaMA 3 70B:     8.17  (post instruction-tuning)
  GPT-3.5:         7.94
  LLaMA 3 8B:      7.01
  Mistral 7B:      6.84
```

LLM-as-Judge Implementation

```
import anthropic, json

client = anthropic.Anthropic()

def llm_judge(question, response, rubric,
              judge_model="claude-sonnet-5") -> dict:
    prompt = f"""You are an expert evaluator. Score this response.

Question: {question}

Response:
{response}

Rubric:
{rubric}

Score each dimension 1-5. Output ONLY JSON:
{{"correctness": N, "completeness": N, "clarity": N,
  "reasoning": "brief explanation", "overall": N}}"""

    resp = client.messages.create(
```

```
        model=judge_model, max_tokens=512,
        messages=[{"role": "user", "content": prompt}],
    )
    text = resp.content[0].text
    start, end = text.find("{"), text.rfind("}") + 1
    return json.loads(text[start:end])

# Example: evaluate SQL generation
question = "Write SQL to find users who placed more than 5 orders last month"
response = """SELECT u.user_id, u.email, COUNT(o.order_id) as order_count
FROM users u JOIN orders o ON u.user_id = o.user_id
WHERE o.created_at >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '1 month')
  AND o.created_at < DATE_TRUNC('month', CURRENT_DATE)
GROUP BY u.user_id, u.email
HAVING COUNT(o.order_id) > 5"""

rubric = """1. Correctness: Does the SQL produce the right result?
2. Completeness: Are all required filters and fields present?
3. Clarity: Is the SQL readable and well-structured?"""

score = llm_judge(question, response, rubric)
# {"correctness": 5, "completeness": 5, "clarity": 5, "overall": 5,
#  "reasoning": "Proper DATE_TRUNC for month boundaries, correct HAVING"}
```

LLM-AS-JUDGE CAVEATS: (1) Positional bias — GPT-4 tends to prefer the first response when comparing two options; randomize order. (2) Self-enhancement bias — a model judging itself scores itself higher. Always use a different, stronger model as judge. (3) Verbosity bias — longer responses score higher regardless of quality; add 'penalise unnecessary verbosity' to your rubric.

Section 7 — Regression Testing: The Production Discipline

A regression suite is a set of hand-curated test cases that must pass after every model update. It catches quality regressions before they reach users. Every production AI system needs one.

```
from dataclasses import dataclass
from typing import Callable
import sqlite3

@dataclass
class EvalCase:
    id: str
    category: str          # "sql_simple", "sql_join", "sql_aggregate"
    question: str
    ground_truth_sql: str
    db_path: str
    notes: str = ""        # WHY this case exists: edge case, past failure


class RegressionSuite:
    def __init__(self, cases: list[EvalCase]):
        self.cases = cases

    def run(self, model_fn: Callable[[str], str]) -> dict:
        results = []
        for case in self.cases:
            predicted = model_fn(case.question)
            try:
                conn = sqlite3.connect(case.db_path)
                pred = set(tuple(r) for r in conn.execute(predicted).fetchall())
                gold = set(tuple(r) for r in conn.execute(case.ground_truth_sql).fetchall())
                correct = pred == gold
                conn.close()
            except Exception:
                correct = False
            results.append({"id": case.id, "category": case.category,
                            "passed": correct, "predicted": predicted})

        by_cat = {}
        for r in results:
            by_cat.setdefault(r["category"], []).append(r["passed"])

        passed = sum(r["passed"] for r in results)
        return {
            "overall_accuracy": passed / len(results),
            "by_category": {k: sum(v)/len(v) for k, v in by_cat.items()},
            "failures": [r for r in results if not r["passed"]],
        }

    def assert_no_regressions(self, model_fn, baseline=0.90):
```

```
        result = self.run(model_fn)
        if result["overall_accuracy"] < baseline:
            raise AssertionError(
                f"REGRESSION: {result['overall_accuracy']:.3f} < {baseline}. "
                f"Failed: {[f['id'] for f in result['failures']]}"
            )
        return result
```

In CI/CD: run the regression suite as a GitHub Actions step on every PR that touches model weights, prompts, or serving config. If the suite fails, block the merge. This is the simplest possible production gate and saves more incidents than any other single practice.

Section 8 — ROUGE and BLEU

ROUGE — For Summarization

```
# ROUGE measures n-gram overlap between generated and reference text
# ROUGE-1: unigram overlap   ROUGE-2: bigram overlap   ROUGE-L: LCS

from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(["rouge1","rouge2","rougeL"], use_stemmer=True)

reference = "The model learns to predict the next token in a sequence."
generated = "The neural network predicts each subsequent token in the text."

scores = scorer.score(reference, generated)
# rouge1: fmeasure=0.57   (7 of 12 words overlap)
# rouge2: fmeasure=0.00   (no bigrams shared)
# rougeL: fmeasure=0.29   (longest common subsequence = 4 words)

Production targets for summarization:
  ROUGE-1 > 40: acceptable    ROUGE-1 > 50: good
  ROUGE-2 > 15: acceptable    ROUGE-2 > 25: good
  ROUGE-L > 35: acceptable    ROUGE-L > 45: good
```

BLEU — For Translation

```
# BLEU measures n-gram precision; corpus-level metric (100+ sentences)
from nltk.translate.bleu_score import corpus_bleu

references = [["the cat sat on the mat".split()]]
hypotheses = ["the cat is sitting on the mat".split()]
score = corpus_bleu(references, hypotheses)  # 0.511

BLEU interpretation for machine translation:
  < 10:    poor quality
  10-29:   understandable but with significant issues
  30-49:   good quality
  50+:     fluent, high-quality translation
```

Task	Use ROUGE?	Use BLEU?	Use Execution Acc?	Use LLM-judge?
SQL generation	No	No	YES (primary)	Yes (secondary)
Code generation	No	No	pass@k (primary)	Yes (secondary)
Summarization	YES	No	No	Yes (secondary)
Translation	Yes	YES	No	Yes (secondary)
Instruction follow	No	No	No	YES (primary)
Math reasoning	No	No	Exact match	Yes (secondary)

Section 9 — lm-evaluation-harness

lm-evaluation-harness (EleutherAI) is the standard tool for running automated benchmarks. It supports 200+ tasks, any HuggingFace model, and vLLM as backend for fast inference.

```
pip install lm-eval

# Run MMLU (5-shot) on fine-tuned SQL model:
lm_eval --model hf \
    --model_args pretrained=~/models/sql-llama-8b-merged,dtype=bfloat16 \
    --tasks mmlu --num_fewshot 5 --batch_size 16 \
    --output_path ./results/sql_model_mmlu.json

# Multiple benchmarks in one command:
lm_eval --model hf \
    --model_args pretrained=meta-llama/Llama-3.1-8B-Instruct,dtype=bfloat16 \
    --tasks mmlu,gsm8k,hellaswag,arc_challenge \
    --num_fewshot 5,8,10,25 \
    --output_path ./results/full_eval.json

# Use vLLM backend for 10x faster evaluation:
lm_eval --model vllm \
    --model_args pretrained=meta-llama/Llama-3.1-8B-Instruct,dtype=bfloat16 \
    --tasks mmlu --num_fewshot 5 --batch_size auto

# Parse results:
import json
with open("./results/full_eval.json") as f:
    results = json.load(f)
print("MMLU:", results["results"]["mmlu"]["acc,none"])
print("GSM8K:", results["results"]["gsm8k"]["exact_match,strict-match"])
```

Before vs After Fine-Tuning Comparison

```
# Run BOTH base and fine-tuned model on MMLU + task benchmark:
# This is your catastrophic forgetting check (Days 9-10)

# Base model:
lm_eval --model hf \
    --model_args pretrained=meta-llama/Llama-3.1-8B-Instruct \
    --tasks mmlu --num_fewshot 5 --output_path ./base_mmlu.json

# Fine-tuned model:
lm_eval --model hf \
    --model_args pretrained=~/models/sql-llama-8b-merged \
    --tasks mmlu --num_fewshot 5 --output_path ./tuned_mmlu.json

# Compare:
base_acc  = json.load(open("base_mmlu.json"))["results"]["mmlu"]["acc,none"]
tuned_acc = json.load(open("tuned_mmlu.json"))["results"]["mmlu"]["acc,none"]
delta = tuned_acc - base_acc
print(f"MMLU delta: {delta:+.3f}")
```

```
# Acceptable: delta > -0.03  (less than 3% drop)
# Action needed: delta < -0.05  (more than 5% drop -> forgetting)
```

Section 10 — FAANG Evaluation in Production

Meta — 5-Tier Evaluation Before Every LLaMA Release

Meta runs a 5-tier evaluation before releasing any LLaMA checkpoint: (1) Automated benchmarks: MMLU, HumanEval, MATH, GSM8K, HumanEval+, MBPP+ — all re-run from scratch for every release candidate. (2) Safety benchmarks: TruthfulQA, BBQ (Bias Benchmark for QA), WinoBias, toxicity. (3) MT-Bench across all 8 categories with GPT-4 as judge. (4) Red-teaming: 500+ adversarial prompts testing jailbreaks, PII leakage, harmful content. (5) Human preference: 200 human annotators rating on helpfulness, harmlessness, honesty. LLaMA 3 70B: 82% MMLU, 81% HumanEval — public, third-party reproducible.

Google — BIG-Bench and HELM

Google Gemini evaluation uses BIG-Bench (204 tasks beyond current LLM training) including logical reasoning, math proofs, multilingual code generation, and 'canary string' detection (did the model memorise this exact string from training data?). HELM (Stanford) standardises evaluation across 7 metrics: accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency — with reproducible prompting protocols. HELM prevents cherry-picking: every metric is reported, not just the good ones.

Amazon — A/B Testing as Ground Truth

Amazon evaluates Bedrock models in production via online A/B testing. A new model version serves 1% of traffic. Click-through rates, task completion rates, and thumbs-up/down signals are collected. Statistical significance tests (Mann-Whitney U) determine whether the new model is better. This is the ground truth that no offline benchmark can replicate — it measures what users actually do, not what they say they prefer. Amazon Bedrock Model Evaluation exposes this as a managed product.

Netflix — Downstream Metric Evaluation

Netflix evaluates LLM output quality through downstream business metrics, not model-level metrics. A 'better' recommendation description model is one whose descriptions drive more content engagement: clicks, watch time, completion rate. Method: generate descriptions with model A and model B for the same 10,000 titles; surface each version to 500K users via holdout; measure engagement delta. Requires 2-4 week experiment windows. This is the gold standard for production LLM evaluation — but only achievable at Netflix's traffic scale.

Microsoft — Responsible AI Evaluation Framework

Microsoft's internal evaluation for Copilot and Azure OpenAI includes four LLM-as-judge dimensions: groundedness (answer supported by retrieved context?), relevance (does it address the question?), coherence (well-structured?), fluency (natural language?). These are implemented as judge prompts and run as CI gates — any model update must score above threshold on all four. Microsoft open-sourced a version as promptflow eval. Also uses DeepEval (open-source) for RAG pipeline evaluation: context precision, context recall, faithfulness, answer relevancy.

Section 11 — Text-to-SQL Thread: Evaluation Harness (Piece 4)

```
#!/usr/bin/env python3
"""Text-to-SQL Evaluation Harness — Capstone Piece 4 of 8."""
import sqlite3, json
from openai import OpenAI
import anthropic

SCHEMA = """
CREATE TABLE users     (user_id INT, email TEXT, country TEXT, created_at DATE);
CREATE TABLE orders    (order_id INT, user_id INT, amount FLOAT,
                        created_at DATE, status TEXT);
CREATE TABLE products  (product_id INT, name TEXT, category TEXT, price FLOAT);
CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, unit_price FLOAT);
"""

BENCHMARK = [
    # Easy: single table
    {"id":"e01","difficulty":"easy",
     "question":"How many users are there?",
     "gold":"SELECT COUNT(*) FROM users"},
    {"id":"e02","difficulty":"easy",
     "question":"List all users from the USA",
     "gold":"SELECT * FROM users WHERE country = 'USA'"},
    # Medium: joins + aggregation
    {"id":"m01","difficulty":"medium",
     "question":"Total revenue per country",
     "gold":"""SELECT u.country, SUM(o.amount) revenue
               FROM users u JOIN orders o ON u.user_id = o.user_id
               GROUP BY u.country ORDER BY revenue DESC"""},
    # Hard: subqueries
    {"id":"h01","difficulty":"hard",
     "question":"Users who spent more than the average order amount",
     "gold":"""SELECT DISTINCT u.email FROM users u
               JOIN orders o ON u.user_id = o.user_id
               WHERE o.amount > (SELECT AVG(amount) FROM orders)"""},
    # ... 46 more in production
]


def run_evaluation(model_url="http://localhost:8000/v1"):
    sql_client = OpenAI(base_url=model_url, api_key="unused")
    judge_client = anthropic.Anthropic()
    conn = sqlite3.connect(":memory:")
    conn.executescript(SCHEMA)
    # Insert sample data...

    exec_results = {"easy": [], "medium": [], "hard": []}
    judge_scores  = []
```

```
    for case in BENCHMARK:
        # Generate SQL
        resp = sql_client.chat.completions.create(
            model="sql-llama-8b-awq",
            messages=[{"role":"system","content":f"Schema:\n{SCHEMA}\nSQL only."},
                      {"role":"user","content":case["question"]}],
            max_tokens=256, temperature=0.05,
        )
        predicted = resp.choices[0].message.content.strip()

        # Execution accuracy
        try:
            pred_rows = set(tuple(r) for r in conn.execute(predicted).fetchall())
            gold_rows = set(tuple(r) for r in conn.execute(case["gold"]).fetchall())
            correct = pred_rows == gold_rows
        except Exception:
            correct = False
        exec_results[case["difficulty"]].append(correct)

    conn.close()

    # Report
    all_results = [v for vals in exec_results.values() for v in vals]
    overall = sum(all_results) / len(all_results)
    print(f"Overall execution accuracy: {overall:.1%}")
    for diff, scores in exec_results.items():
        print(f"  {diff}: {sum(scores)/len(scores):.1%} ({len(scores)} cases)")
    return overall


if __name__ == "__main__":
    run_evaluation()
```

CAPSTONE PIECES STATUS: Piece 1 (SQLOutput schema, Day 6) DONE. Piece 2 (Schema RAG, Week 3) UPCOMING. Piece 3 (Self-correcting agent, Week 5) UPCOMING. Piece 4 (Evaluation harness, Day 12) DONE. Piece 5 (Multi-agent architecture, Week 8) UPCOMING. Piece 6 (Security layer, Week 10) UPCOMING. Piece 7 (LLMOps dashboard, Week 11) UPCOMING. Piece 8 (Full capstone, Week 12) UPCOMING.

Section 12 — Architect's Lens: The Evaluation Pyramid

```
EVALUATION PYRAMID — apply from bottom to top:

    +---------------------------------------+
    | HUMAN EVALUATION (Gold)               |  Expensive, slow; use for final
    | Chatbot Arena, A/B, red-teaming       |  sign-off and unknown failure modes
    +---------------------------------------+
    | LLM-AS-JUDGE (Silver)                 |  Fast, scalable; daily CI gate;
    | MT-Bench, custom rubrics              |  catches quality regressions
    +---------------------------------------+
```

```
    | AUTOMATED BENCHMARKS (Bronze)         |  Cheap, reproducible; every run;
    | MMLU, HumanEval, Spider, GSM8K        |  before/after FT comparison
    +---------------------------------------+
```

```
METRIC SELECTION BY TASK:
  Task                Primary Metric       Secondary
  SQL generation      Execution accuracy   LLM-judge (SQL quality)
  Code generation     pass@1 (HumanEval)   pass@10 (diversity)
  Instruction follow  MT-Bench             Human preference
  World knowledge     MMLU                 TruthfulQA
  Summarization       ROUGE-L + ROUGE-2    Human quality rating
  Translation         BLEU (corpus)        Human fluency
  Math reasoning      GSM8K exact match    CoT pass@1

BENCHMARK CONTAMINATION WARNING:
  Many closed models are suspected to have MMLU/HumanEval in training data.
  Always supplement with:
  - HumanEval+ (80x more test cases; harder to memorise)
  - Your own private benchmark (never published = never contaminated)
  - MT-Bench (conversational; harder to memorise than MCQ)
  - Task-specific execution accuracy (canonical ground truth)

WHEN TO EVALUATE:
  Before fine-tuning:    baseline on MMLU + task metric
  After fine-tuning:     re-run BOTH (task improved AND general didn't regress)
  Model comparison:      same benchmark, same prompting format, same seed
  Every PR (CI gate):    regression suite (execution accuracy >= baseline - 2%)
  Release sign-off:      full pyramid: benchmarks + LLM judge + human eval
```

Section 13 — Hands-On Exercises

Exercise 1: Before/After Fine-Tuning MMLU Comparison

```
# Run on base model:
lm_eval --model hf \
    --model_args pretrained=meta-llama/Llama-3.1-8B-Instruct,dtype=bfloat16 \
    --tasks mmlu,gsm8k --num_fewshot 5,8 --batch_size 8 \
    --output_path ./results/base.json

# Run on your SQL fine-tuned model:
lm_eval --model hf \
    --model_args pretrained=~/models/sql-llama-8b-merged,dtype=bfloat16 \
    --tasks mmlu,gsm8k --num_fewshot 5,8 --batch_size 8 \
    --output_path ./results/tuned.json

# Compare and decide:
import json
base  = json.load(open("./results/base.json"))["results"]
tuned = json.load(open("./results/tuned.json"))["results"]

for task in ["mmlu", "gsm8k"]:
    key = "acc,none" if task == "mmlu" else "exact_match,strict-match"
    b = base[task][key]
    t = tuned[task][key]
    delta = t - b
    status = "OK" if delta > -0.03 else "FORGETTING WARNING" if delta > -0.05 else "FAIL"
    print(f"{task}: base={b:.3f} tuned={t:.3f} delta={delta:+.3f} [{status}]")
```

Exercise 2: Build a 5-Case Regression Suite

```
# Minimum viable regression suite for your SQL model:
import sqlite3

def setup_db():
    conn = sqlite3.connect(":memory:")
    conn.execute("CREATE TABLE users (user_id INT, email TEXT, country TEXT)")
    conn.execute("CREATE TABLE orders (order_id INT, user_id INT, amount FLOAT)")
    conn.execute("INSERT INTO users VALUES (1,'a@x.com','USA'),(2,'b@x.com','UK')")
    conn.execute("INSERT INTO orders VALUES (1,1,100.0),(2,1,200.0),(3,2,150.0)")
    conn.commit()
    return conn

REGRESSION_CASES = [
    ("COUNT all users",
     "SELECT COUNT(*) FROM users",
     "SELECT COUNT(*) FROM users"),
    ("Users from USA",
     "SELECT * FROM users WHERE country = 'USA'",
     "SELECT * FROM users WHERE country = 'USA'"),
    ("Total orders per user",
     "Get order count per user",
```

```
     """SELECT user_id, COUNT(*) as cnt FROM orders
        GROUP BY user_id ORDER BY cnt DESC"""),
    ("High value orders",
     "Orders over $150",
     "SELECT * FROM orders WHERE amount > 150"),
    ("Users with orders",
     "Email of users who placed at least one order",
     """SELECT DISTINCT u.email FROM users u
        JOIN orders o ON u.user_id = o.user_id"""),
]

conn = setup_db()
passed = 0
for name, question, gold_sql in REGRESSION_CASES:
    try:
        pred = text_to_sql(question)  # your model function
        pred_r = set(tuple(r) for r in conn.execute(pred).fetchall())
        gold_r = set(tuple(r) for r in conn.execute(gold_sql).fetchall())
        ok = pred_r == gold_r
    except Exception:
        ok = False
    passed += ok
    print(f"  {'PASS' if ok else 'FAIL'}: {name}")
print(f"Result: {passed}/{len(REGRESSION_CASES)}")
```

Exercise 3: Identify the Right Metric

Task	Primary Metric	Tool
Fine-tuned SQL model quality	Execution accuracy (Spider)	Custom harness
Did fine-tuning cause forgetting?	MMLU delta	lm-eval
Instruction following quality	MT-Bench (GPT-4 judge)	fastchat
Code generation	pass@1 (HumanEval+)	lm-eval
Summarization quality	ROUGE-2 + ROUGE-L	rouge-score
Production quality drift	A/B test or regression suite	Custom CI

Summary — Day 12 Key Concepts

Concept	One-Line Definition
MMLU	57-subject MCQ; world knowledge; random=25%, GPT-4=86%
HumanEval	164 Python problems; scored by pass@k; unit test execution
HumanEval+	80x more test cases; harder to cheat than original HumanEval
GSM8K	Grade school math; tests multi-step chain-of-thought reasoning
Spider	Text-to-SQL; 10K questions; execution accuracy is the metric
Execution accuracy	SQL correct iff result rows equal ground truth rows
MT-Bench	80 multi-turn convos; GPT-4 judges 1-10; 8 categories
LLM-as-judge	Stronger model evaluates weaker; rubric-based; catches nuance
pass@k	P(at least 1 of k samples passes all unit tests)
ROUGE-L	Longest common subsequence overlap; for summarization
BLEU	N-gram precision; for translation; not for SQL or code
lm-evaluation-harness	EleutherAI; 200+ benchmarks; vLLM backend for fast eval
Regression suite	Curated test cases; CI gate; catch regressions before deploy
Benchmark contamination	Model trained on test data -> inflated scores; use private evals

TOMORROW — Day 13: Efficiency Deep Dive

KV Cache tuning, speculative decoding mechanics (draft models, acceptance rate math), Flash Decoding, quantization-aware serving, and throughput optimization strategies. How to squeeze maximum tokens/second from the same hardware you already have.

Eagle Eye Addendum — Day 12: RAGAS (RAG Evaluation Framework)

RAGAS (Retrieval Augmented Generation Assessment) is an open-source framework for evaluating RAG pipelines end-to-end. It computes four key metrics automatically using an LLM as judge.

```
from ragas import evaluate
from ragas.metrics import (faithfulness, answer_relevancy,
                           context_precision, context_recall)
from datasets import Dataset

# RAGAS evaluates 4 dimensions of RAG quality:
# 1. faithfulness     — does the answer contradict the retrieved context? (0–1)
# 2. answer_relevancy — does the answer address the question? (0–1)
# 3. context_precision— are retrieved chunks ranked correctly? (0–1)
# 4. context_recall   — does the retrieved context cover the answer? (0–1)

eval_data = Dataset.from_dict({
    "question":  ["What is LoRA?"],
    "answer":    ["LoRA adds low-rank matrices to frozen model weights."],
    "contexts":  [["LoRA decomposes weight updates into A×B matrices..."]],
    "ground_truth":["LoRA reduces trainable parameters via low-rank decomposition."],
})

result = evaluate(eval_data,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall])

# result["faithfulness"]      → 0.92  (answer matches context)
# result["answer_relevancy"]  → 0.87  (answer addresses question)
# result["context_precision"] → 0.78  (retrieved chunks are ranked well)
# result["context_recall"]    → 0.95  (context covers the answer)

# Production SLOs (Day 15-17 RAG pipeline):
# faithfulness > 0.90, answer_relevancy > 0.85
```